

1 Assignment: 1

► How to add and subtract two n -bit integers?

basic addition operations:

$$0 + 0 = 0, \quad 1 + 0 = 1, \quad 0 + 1 = 1, \quad 1 + 1 = \underset{\substack{\downarrow \\ \text{(Carry)}}}{10}$$

To simply add two 1-bit numbers we need a **half adder circuit**.

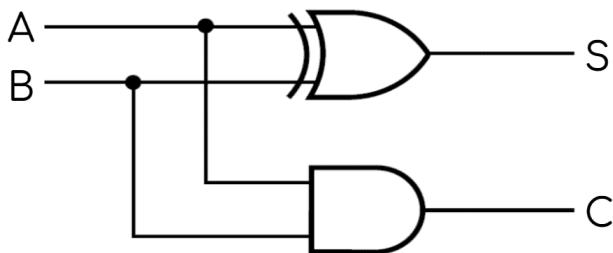
Equations:

$$S = \bar{A}B + A\bar{B} = A \oplus B$$

$$C = A \cdot B$$

Truth Table:

A	B	Sum(S)	Carry out(C_{out})
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



To add two 2-bit numbers we need a **full adder circuit**.

Equations:

$$S = \bar{C}_{in}\bar{A}B + \bar{C}_{in}A\bar{B} + C_{in}\bar{A}\bar{B} + C_{in}AB$$

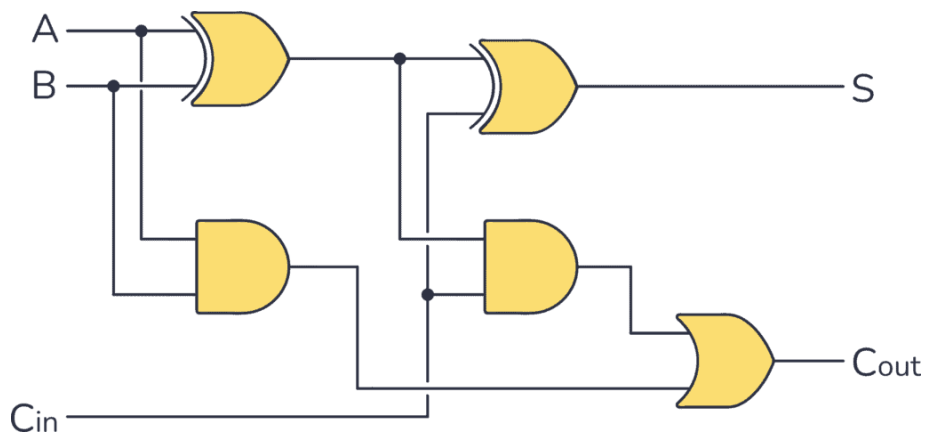
$$= A \oplus B \oplus C$$

$$C_{out} = \bar{C}_{in}AB + C_{in}\bar{A}B + C_{in}A\bar{B} + C_{in}AB$$

$$= C_{in}(A \oplus B) + AB$$

Truth Table:

C_{in}	A	B	Sum(S)	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Add n -bit integers:

We want $(n - 1)$ number of full adders and 1 half adder.

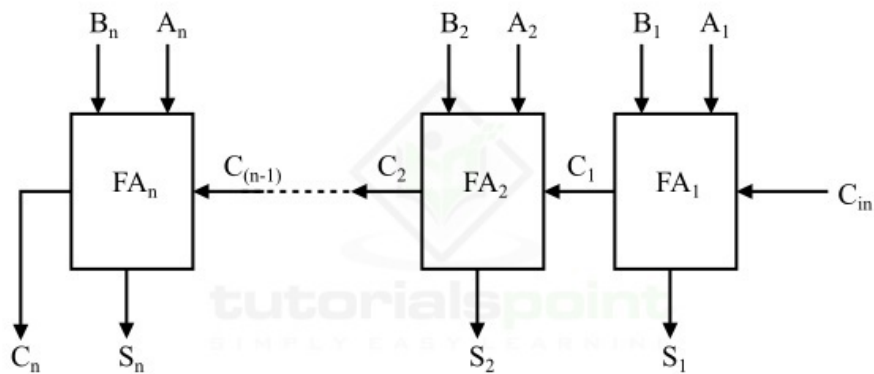


Figure 2 - N-Bit Parallel Adder

► Subtraction Rules

$$0 - 0 = 0, \quad 0 - 1 = \underset{\substack{\downarrow \\ \text{borrow}}}{11}, \quad 1 - 0 = 1, \quad 1 - 1 = 0$$

Subtraction of two 1-bit binary numbers:

We use a **Half Subtractor**.

Equations:

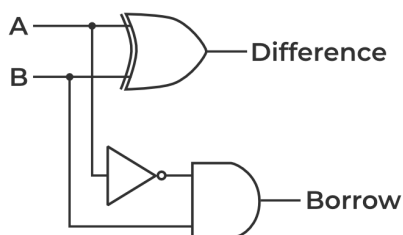
$$D = A \oplus B$$

$$B_{\text{out}} = \bar{A}B$$

Truth Table:

A	B	Difference (D)	Borrow (B_{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Circuit Diagram:



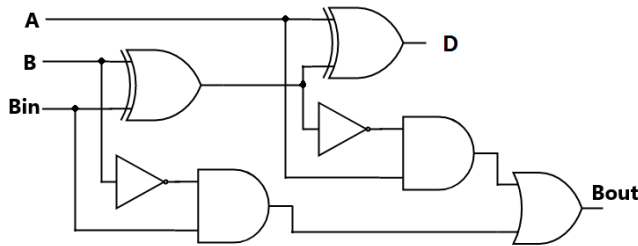
Full Subtractor: Used for subtraction between two 1-bit binary numbers and considers the borrow of the previous bit.

Equations:

$$D = A \oplus B \oplus B_{\text{in}}$$

$$B_{\text{out}} = \bar{A}B + B_{\text{in}}(A \oplus B)'$$

Circuit Diagram:



Truth Table:

A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2's Complement Subtraction

Subtraction of A by B is obtained by taking the 2's complement of B and adding it to A .

- The 2's complement of B is obtained by taking the 1's complement of B and adding 1 to the LSB (Least Significant Bit).

Parallel Adder / Subtractor Circuit

We use n Full Adders. The mode input control line M is connected with the carry input of the LSB (C_{in}). The control line decides whether addition or subtraction is performed.

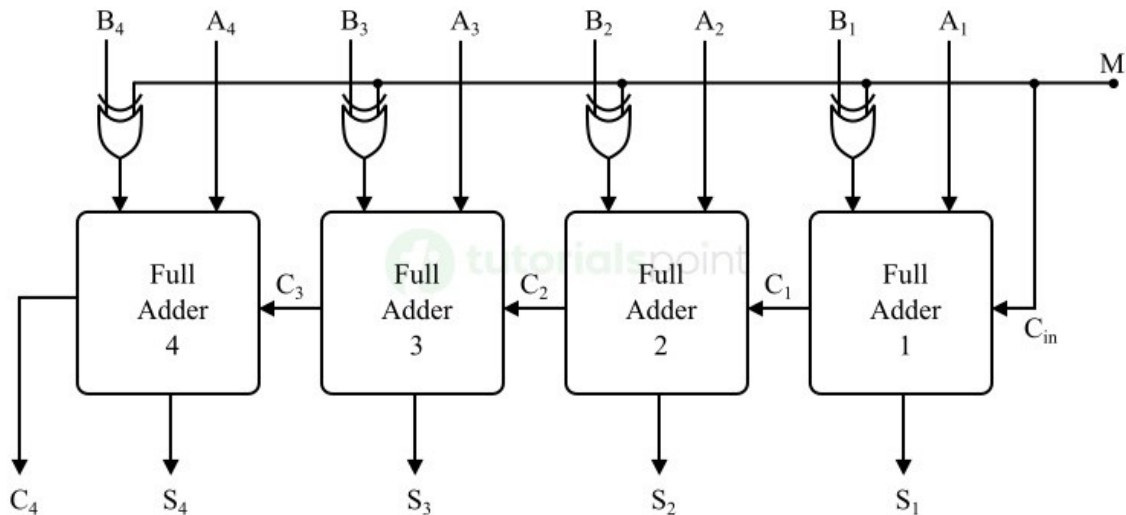


Figure 1: 4-bit Binary Parallel Adder / Subtractor Circuit

Working Operation:

1. When $M = 1$ (Subtractor Mode):

- $B \oplus 1 = \bar{B}$ (produces 1's complement of B).

- The carry input $C_{\text{in}} = 1$ is added to the LSB.
- Hence, complemented B along with 1 added via carry input yields $A + (2\text{'s complement of } B)$.
- Thus, the **subtraction operation** ($A - B$) is performed.

2. When $M = 0$ (Adder Mode):

- $B \oplus 0 = B$.
- The carry input $C_{\text{in}} = 0$.
- Hence, the normal **addition operation** ($A + B$) is performed.

2 Assignment: 2

* C code to find GCD of two integers :

Let a, b ($b \neq 0$) be two integers (+ve integers).

By Euclidean Algorithm, $\exists$ unique integers q and r such that

$$a = bq + r \quad \text{where } 0 \leq r < b$$

So, $\text{gcd}(a, b) = \text{gcd}(b, r)$

We repeat the process until after division remainder is zero.

$$r_0 = q_1 r_1 + r_2 \quad (r_2 < r_1)$$

$$r_1 = q_2 r_2 + r_3 \quad (r_3 < r_2)$$

$$r_0 = q_1(q_2 r_2 + r_3) + r_2$$

$$= (q_1 q_2 + 1)r_2 + q_1 r_3$$

$$r_0 > (q_1 q_2 + 1)r_2 > 2r_2$$

$$\implies r_2 < \frac{r_0}{2}$$

Remainder is cut in half at least every two steps. So, for n -bit integer, as max value is 2^n , algorithm terminates in **max $2n$ steps**.

C Program to Find GCD of Two Integers

```

1 unsigned int find_gcd(unsigned int a, unsigned int b) {
2     while (b != 0) {
3         unsigned int rem = a % b;
4         a = b;
5         b = rem;
6     }
7     return a;
8 }

```

To calculate the time:

```
1 #include <stdio.h>
2 #include <time.h>
3
4 unsigned int find_gcd(unsigned int a, unsigned int b) {
5     while (b != 0) {
6         unsigned int rem = a % b;
7         a = b;
8         b = rem;
9     }
10    return a;
11 }
12
13 int main() {
14     clock_t t1, t2;
15     int i, repcount;
16     float timeTaken;
17
18     unsigned int x,y;
19     unsigned int result;
20     print("Enter two positive integers: ");
21     scanf("%u %u",&x,&y);
22     repcount = 1000000;
23
24     t1 = clock();
25     for (i = 0; i < repcount; i++) {
26         result = find_gcd(x, y);
27     }
28
29     t2 = clock();
30
31     timeTaken = (((float)(t2 - t1) / (float)CLOCKS_PER_SEC) / (float)repCount);
32
33     printf("GCD of %u and %u is: %u\n", x, y, result);
34     printf("Time taken = %f seconds\n", timeTaken);
35
36     return 0;
37 }
```